

Summary

This topic synthesizes the core principles of algorithmic complexity analysis as presented in the source context. The analysis focuses on two primary areas: the evaluation of iterative loops (both independent and dependent nested loops) and the foundational construction of recurrence relations for recursive algorithms.

Critical Takeaways:

- **Loop Dynamics:** The complexity of a loop is determined by its increment/decrement logic; linear progression results in $O(n)$, while geometric progression (doubling or halving) results in $O(\log n)$.
- **Nested Loop Calculation:** For independent nested loops, total complexity is the product of the individual loops. For dependent loops, complexity is derived through the summation of terms, often involving Arithmetic Progressions (AP).
- **Summation Method:** A formal mathematical approach using $\sum$ notation can precisely determine the exact number of executions in complex nested structures.
- **Recurrence Construction:** Recursive functions are analyzed by expressing the total running time $T(n)$ as a function of the recursive call $T(n-k)$ plus the non-recursive (local) work performed in each call.

1. Iterative Loop Analysis

The source establishes a baseline for determining the time complexity of iterative structures by examining how many times a core statement (e.g., a printf or a specific operation) executes based on the loop's constraints.

1.1 Linear and Logarithmic Foundations

Complexity is viewed through the lens of $n \rightarrow \infty$. In this context, constant additions or minor variations (like n vs. $n+1$) are often simplified in asymptotic notation.

Loop Structure	Step Logic	Approximate Executions	Complexity Class
$(i = 1) \text{ to } (n); (i++)$	Increment by 1	(n)	$(O(n))$
$(i = 1) \text{ to } (n); (i = i \times 2)$	Double each step	$(\log_2 n + 1)$	$(O(\log n))$
$(i = n) \text{ to } 1; (i = i / 2)$	Halve each step	$(\log_2 n)$	$(O(\log n))$

1.2 Independent Nested Loops

In independent nested loops, the inner loop executes the same number of times regardless of the outer loop's current iteration value. The total complexity is the product of the iterations of the outer and inner loops.

- Example 1: An outer loop running 3 times and an inner loop running (n) times results in $3n$ executions, which simplifies to $(\theta(n))$.
- Example 2: An outer loop running (n) times and an inner loop running (n) times results in $(n \times n = n^2)$ executions, or $(O(n^2))$.
- Example 3: An outer loop running $(\log n)$ times and an inner loop running (n) times results in $(n \log n)$ complexity.

2. Advanced Nested Loop Patterns

Analysis becomes more complex when the inner loop's boundaries are dependent on the outer loop's variable.

2.1 Dependent Nested Loops

When the inner loop depends on the outer loop variable (i), the total executions must be calculated by summing the iterations for each value of (i).

- Pattern 1: ($j = 1$) to (i)
If the outer loop runs ($i = 1$) to (n) and the inner loop runs ($j = 1$) to (i), the total steps follow the sum of the first (n) natural numbers:
[
 $1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$
]
The complexity is $(O(n^2))$.
- Pattern 2: ($j = i$) to ($2i$)
For a loop where (i) goes from 1 to (n) and (j) goes from (i) to ($2i$), the number of iterations per step is calculated using the formula:
[
 $\text{Last} - \text{First} + 1$
]
Summing the steps:
[
 $2 + 3 + 4 + \dots + (n+1)$
]
Using the Arithmetic Progression (AP) sum formula:
[
 $\frac{n}{2} \times (\text{First Term} + \text{Last Term}) = \frac{n}{2} \times (2 + n + 1)$
]

Result:

$$\left[\frac{n}{2} \times (n + 3) \right]$$

Which simplifies to $(O(n^2))$.

2.2 The Summation Method

The summation method is a formal approach to solving dependent nested loops by converting loop boundaries into mathematical summations.

Process Example:

1. Inner Loop: Represent the inner loop as $(\sum_{j=i}^n 1)$. The number of steps is $(3i - i + 1 = 2i + 1)$.
2. Outer Loop: Wrap the result in the outer loop's summation:
$$\left[\sum_{i=1}^n (2i + 1) \right]$$
3. Expansion:
$$\left[2 \sum_{i=1}^n i + \sum_{i=1}^n 1 \right]$$
4. Solution:
$$\left[2 \times \frac{n(n+1)}{2} + n = n^2 + n + n = n^2 + 2n \right]$$
5. Complexity: $(O(n^2))$.

3. Recurrence Relations for Recursive Algorithms

Recurrence relations are used to model the running time of recursive functions. The total time $(T(n))$ is the sum of the time taken for recursive calls plus the time taken by non-recursive statements within the function.

3.1 Components of a Recurrence Relation

- Base Case: The condition where recursion stops (e.g., $(T(0) = 1)$ or $(T(1) = 1)$).
- Recursive Step: The call to the function with a reduced input size (e.g., $(T(n-1))$).
- Local Work: The complexity of the operations performed in the current call (constant (C) , linear (n) , or logarithmic $(\log n)$).

3.2 Construction Examples

Function Structure	Non-Recursive Work	Recurrence Relation
Single recursive call ($f(n-1)$)	Constant (C)	($T(n) = T(n-1) + C$)
Single recursive call ($f(n-1)$)	Linear Loop ($O(n)$)	($T(n) = T(n-1) + cn$)
Single recursive call ($f(n-1)$)	Logarithmic Loop ($O(\log n)$)	($T(n) = T(n-1) + c \log n$)
Two recursive calls to ($f(n-1)$)	Linear Loop ($O(n)$)	($T(n) = 2T(n-1) + cn$)

4. Key Instructional Insights

The following insights summarize the instructor's perspective on the methodology of algorithm analysis:

- Precision in Notation: While (O) (Upper Bound) is common, (θ) (Tight Bound) is often the more accurate notation when best and worst cases are identical.
- Handling Constants: In asymptotic analysis, constants (like ($10n$) vs. (n)) and lower-order terms (like (n) in ($n^2 + n$)) are dropped.
- Equivalence of Direction: Moving from 1 to (n) by doubling is computationally equivalent to moving from (n) to 1 by halving; both are ($\log n$) processes.
- Complexity as a Function of Input: Running time must always be viewed as ($T(n)$), where (n) represents the input size.

"Everything is easy if you understand the basics. In complexity, we don't talk about small differences like 1 or 2; we are 'rich people' looking at (n) tending toward infinity."